

A Project Report On

Franxx

Submitted in partial fulfillment of the requirement for the
award of the degree

Master of Computer Applications
(MCA)

Academic Year 2026 – 27

Savan Detroja (92500584067)
Jevin Parmar (92500584073)
Aryan Kapadiya (92500584093)

Internal Guide

Dr. Jaypalsinh Gohil



Marwadi
University
Marwadi Chandarana Group



Rajkot-Morbi Road, At & PO :Gauridad, Rajkot 360 003. Gujarat. India.



Marwadi
University
Marwadi Chandarana Group



Faculty of Computer Applications (FoCA)

Certificate

**This is to certify that the project work entitled *Franxx*
submitted in partial fulfillment of the requirement for
the award of the degree of**

**Master of Computer Applications (MCA)
of the**

Marwadi University

is a result of the bonafide work carried out by

Savan Detroja (92500584067)

Jevin Parmar (92500584073)

Aryan Kapadiya (92500584093)

during the academic year 2026-27

Faculty Guide

HOD

Dean

DECLARATION

We hereby declare that this project work entitled Franxx is a record done by us.

We also declare that the matter embodied in this project is genuine work done by us and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Place: MARWADI UNIVERSITY

Date: 10/07/2026

Savan Detroja (92500584067)

Signature: _____

Jevin Parmar (92500584073)

Signature: _____

Aryan Kapadiya (92500584093)

Signature: _____

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us. No serious and lasting achievement or success one can ever achieve without the help of friendly guidance and co-operation of so many people involved in the work.

We are very thankful to our guide **Dr. Jaypalsinh Gohil**, the person who makes us to follow the right steps during our project work. We express our deep sense of gratitude to for his /her guidance, suggestions and expertise at every stage. A part from that his/her valuable and expertise suggestion during documentation of our report indeed help us a lot.

Thanks to our friend and colleague who have been a source of inspiration and motivation that helped to us during our project work.

We are heartily thankful to the Dean of our department **Dr. Rijwan Khan** sir and HoD **Dr. Sunil Bajaja** sir for giving us an opportunity to work over this project and for their end-less and great support to all other people who directly or indirectly supported and help us to fulfil our task.

Savan Detroja (92500584067)	Signature:_____
Jevin Parmar (92500584073)	Signature:_____
Aryan Kapadiya (92500584093)	Signature:_____

CONTENTS

Chapters	Particulars	Page No.
1	Introduction to Project Definition	1
2	PREAMBLE	2
2.1	Module description	3
3	REVIEW OF LITERATURE Analyze, Study and Compare the similar types of the systems/applications and highlights major findings	5
4	TECHNICAL DESCRIPTION	7
4.1	Hardware Requirement	
4.2	Software Requirement	
5	SYSTEM DESIGN AND DEVELOPMENT	
5.1	• - Flow Chart	8
5.2	• - Data Flow Diagram	9
5.3	• - Class Diagram	10
5.4	• - Use Case Diagram	11
5.5	• - Sequential Diagram	12
5.6	• - Activity Diagram	13
5.7	• Screen Design	14
5.8	• Database Design / File Structure	15
6	SYSTEM TESTING	23
7	CONCLUSION	28
8	LEARNING DURING PROJECT WORK	29
8.1	Future Enhancement	30
9	BIBLIOGRAPHY	31
9.1	Online References	
9.2	Offline References	

Table Index

Sr No.	Table No.	Particulars	Page No.
1	Table 4.1	Hardware Requirement	7
2	Table 4.2	Software Requirement	7
3	Table 5.8	Users Collection	20
4	Table 5.8.1	Chat Collection	20
5	Table 5.8.2	Messages Collection	21
6	Table 5.8.3	Status Collection	21
7	Table 5.8.4	Notifications Collection	22
8	Table 5.8.5	Settings Collection	22
9	Table 6	Registration & Login Testing	23
10	Table 6.1	Private Chat Testing	23
11	Table 6.2	Chat Testing	23
12	Table 6.3	User Search & Connection Testing	24
13	Table 6.4	User Profile Testing	25
14	Table 6.5	Status Testing	25
15	Table 6.6	Media & File Sharing Testing	26
16	Table 6.7	Notification Testing	26
17	Table 6.8	Settings Testing	27
18	Table 6.9	Admin Panel Testing	27

Figure Index

Sr No.	Figure No.	Particulars	Page No.
1	Figure 5.1	Flow Chart	8
2	Figure 5.2	Data Flow Diagram	9
3	Figure 5.3	Class Diagram	10
4	Figure 5.4	Use Case Diagram	11
5	Figure 5.5	Sequential Diagram	12
6	Figure 5.6	Activity Diagram	13
7	Figure 5.7	User Search Page	14
8	Figure 5.7.1	Status page	15
9	Figure 5.7.2	Media and Gallery page	17
10	Figure 5.7.3	Settings page	17
11	Figure 5.7.4	Profile page	18
12	Figure 5.7.5	Admin Dashboard page	18
13	Figure 5.7.6	User Panel page	19

1.Introduction Project Description

Communication has become one of the most essential parts of modern digital life. Existing messaging applications provide users with secure and fast communication, but developing such a platform helps students understand full-stack application development and real-time communication technologies.

FRANXX is designed as a modern messaging platform inspired by popular applications such as WhatsApp, Discord, Telegram, and Messenger. The application provides a secure environment where registered users can communicate with other users through private chats or group conversations.

The system uses MongoDB for storing user information, chat history, media details, and group information. Express.js and Node.js handle server-side operations, authentication, and APIs, while React.js provides an interactive and responsive user interface. Socket.IO enables real-time message delivery without requiring users to refresh the page.

The application also includes an administrator module that allows administrators to manage users, monitor chats, handle reports, and maintain the overall system.

The system is designed to be scalable, user-friendly, and secure while following modern software development practices.

2. PREAMBLE

FRANXX is a web-based real-time chat application developed using the MERN stack. It allows users to register, securely log in, search and connect with other users, send private messages, create group chats, share media and files, and post temporary statuses. The system also provides notifications, user profiles, settings, and an admin panel for managing users and application activities. The main aim of FRANXX is to provide a simple, secure, and user-friendly platform for real-time communication.

2.1 Module Description

1. User Authentication Module

- User registration, login and logout.
- Secure password encryption and JWT authentication.
- Protected routes and session management.

2. Dashboard Module

- Displays recent chats, online users and latest messages.
- Provides quick access to search, profile and settings.
- Acts as the main screen after successful login.

3. Private Chat Module

- Real-time one-to-one messaging between users.
- Supports text messages, emojis, images and file sharing.
- Shows message time, seen status, chat history and shared media.

4. Group Chat Module

- Users can create groups and communicate with multiple members.
- Supports adding/removing members, group image and description.
- Group admin can manage members and group information.

5. User Profile Module

- View and edit profile picture, username, email and about information.
- Shows online status and last seen.
- Users can view their own profile as well as other users' profiles.

6. User Search and Connection Module

- Search registered FRANXX users by name or username.
- View searched user's profile.
- Send connection/friend request and start a private chat after connection.

7. Followers and Following Module

- Manage followers and following users.
- Accepted connections are displayed in the user's network.
- Connected users can easily access each other from the chat section.

8. Status Module

- Upload temporary text or image status.
- View status updates shared by connected/following users.
- Users can also delete their own status.

9. Media and File Sharing Module

- Share images, videos, documents and other supported files in chats.
- View shared media and files from conversations.
- Download shared files when required.

10. Notification Module

- Provides notifications for new messages and user activities.
- Shows friend/connection requests and group-related notifications.
- Helps users stay updated without manually checking each section.

11. Settings Module

- Edit account information and change password.
- Manage notification, privacy, security and theme settings.
- Provides logout and other account management options.

12. Admin Module

- Provides a separate dashboard for administrators.
- Manage users, chats, groups and reported activities.
- View application statistics, system logs and overall platform activity.

3.REVIEW OF LITERATURE

Analysis, Study and Comparison of Similar Systems A literature review was conducted to study existing messaging and social communication platforms. The analysis helped to understand their important features, strengths and limitations. These systems were studied as references for developing the FRANXX.

1. WhatsApp

Description:

WhatsApp is a popular messaging application that allows users to communicate through private and group chats.

Strengths:

- Fast real-time private and group messaging.
- Supports images, videos, documents and status sharing.
- Provides online status, last seen and message status.

Limitations:

- Mainly depends on phone numbers for connecting users.
- Provides limited public user-search functionality.
- Limited community/profile discovery compared to social platforms.

2. Telegram

Description:

Telegram is a cloud-based messaging application that provides private chats, groups and media sharing.

Strengths:

- Fast real-time messaging.
- Supports large groups and file sharing.
- Users can be searched using usernames.

Limitations:

- Contains many advanced features that can make the application complex.
- Some privacy and messaging features may be difficult for new users to understand.

3. Discord

Description:

Discord is a communication platform that provides private messaging, group communication and community-based servers.

Strengths:

- Real-time communication.
- Supports private and group conversations.
- Provides online status and media/file sharing.

Limitations:

- Mainly designed around communities and servers.
- Interface can be complex for users who only need simple messaging.
- Many features are unnecessary for basic personal communication.

4. Instagram

Description:

Instagram is a social networking platform that also provides private messaging, user profiles, followers and temporary status/story sharing.

Strengths:

- Users can search for other users easily.
- Provides followers and following functionality.
- Supports private messaging and story/status sharing.

Limitations:

1. Mainly focused on social media and content sharing.
2. Messaging is not the primary purpose of the platform.
3. Contains many features unrelated to a dedicated chat application.

4. TECHNICAL DESCRIPTION

4.1 Hardware Requirement

Component	Minimum Requirement
Processor	Intel Core i3 or AMD equivalent
RAM	4 GB (8 GB Recommended)
Hard Disk	20 GB Free Space
Internet Connection	Broadband/Wi-Fi

Table 4.1 Hardware Requirement

4.2 Software Requirement

Software	Requirement
Operating System	Windows 10/11, Linux, or macOS
Frontend	React.js, HTML5, CSS3, JavaScript, Bootstrap/Tailwind CSS
Backend	Node.js, Express.js
Database	MongoDB
Runtime Environment	Node.js (v18 or above)
IDE	Visual Studio Code
API Testing Tool	Postman
Version Control	Git and GitHub
Web Browser	Google Chrome, Microsoft Edge, Mozilla Firefox
Package Manager	npm (Node Package Manager)

Table 4.2 Software Requirement

5. SYSTEM DESIGN AND DEVELOPMENT

5.1 Flowchart

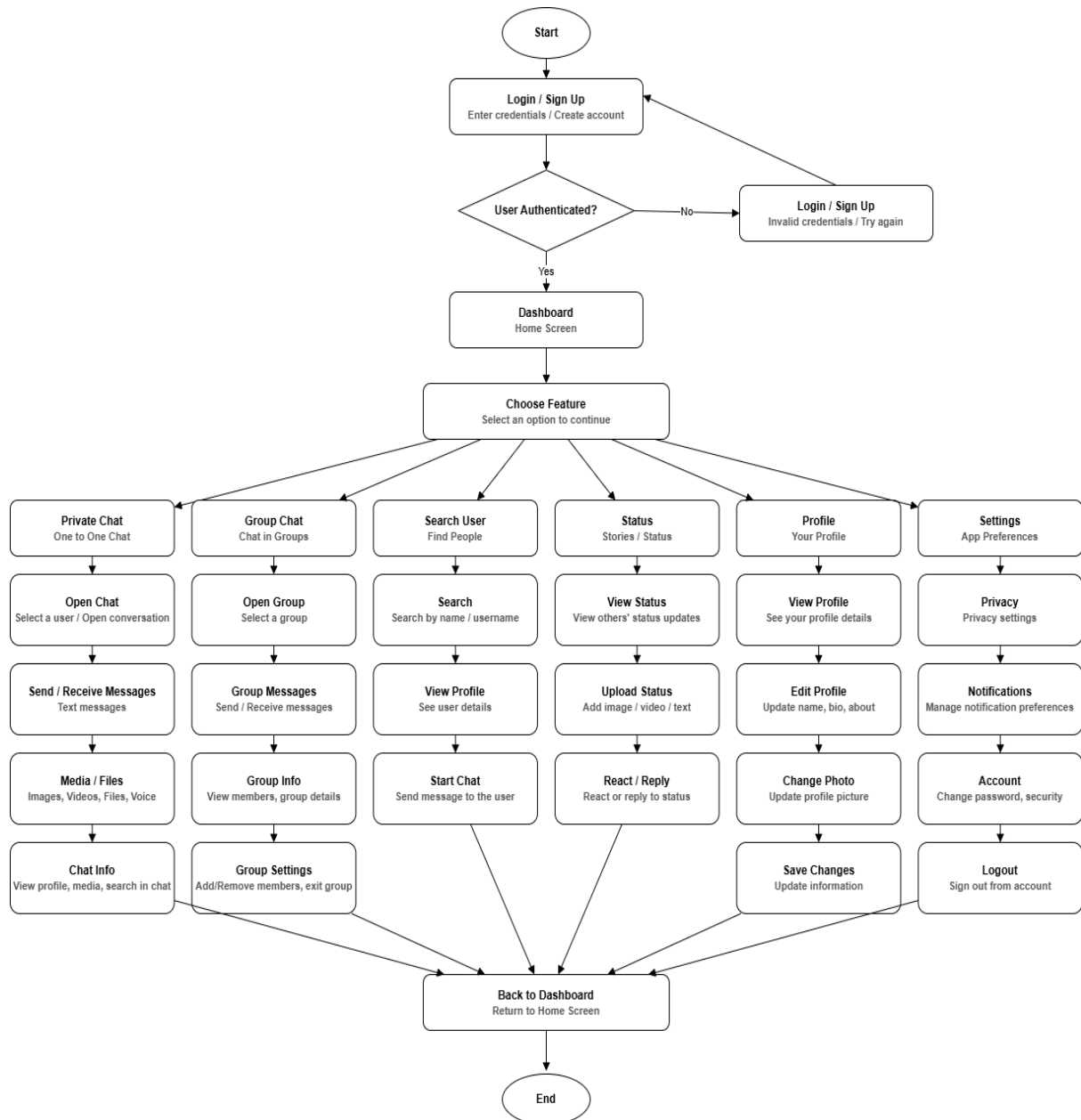
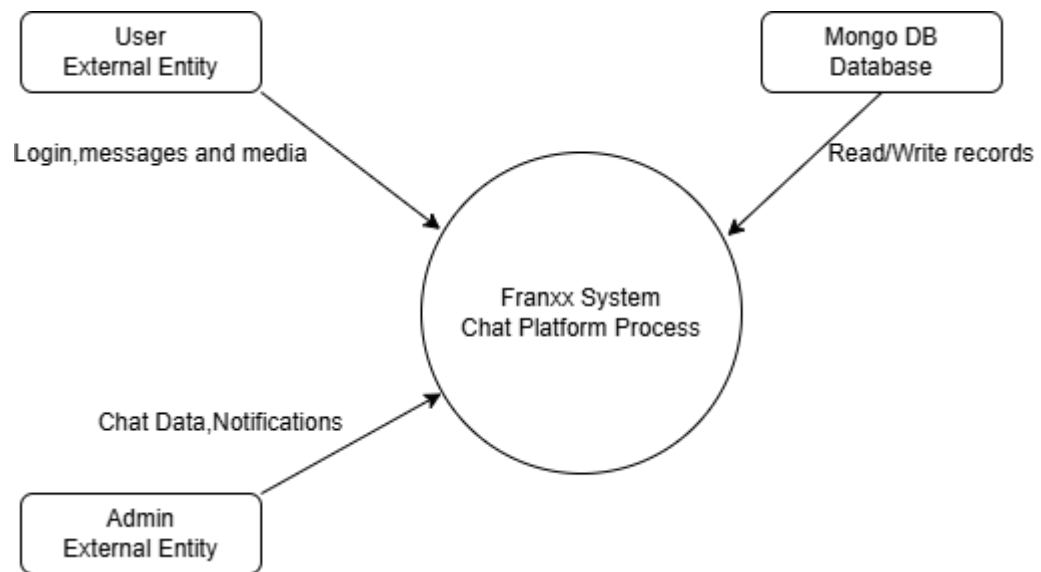


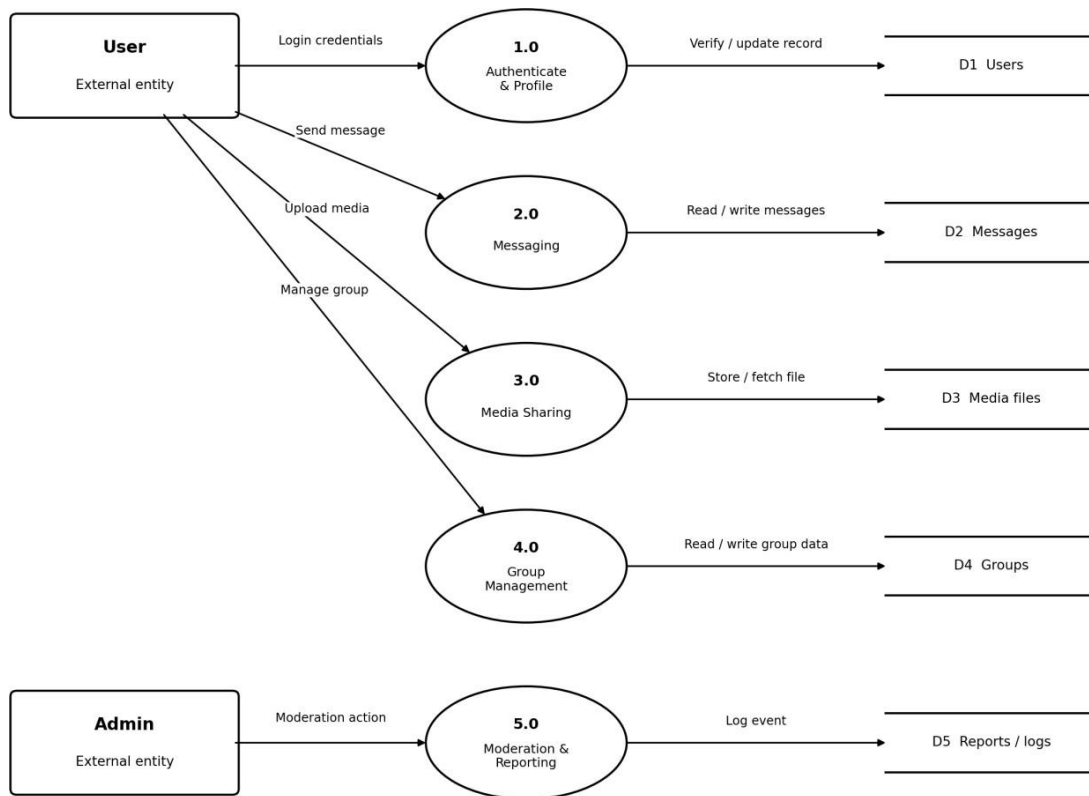
Figure 5.1 Flowchart

5.2 Data Flow Diagram

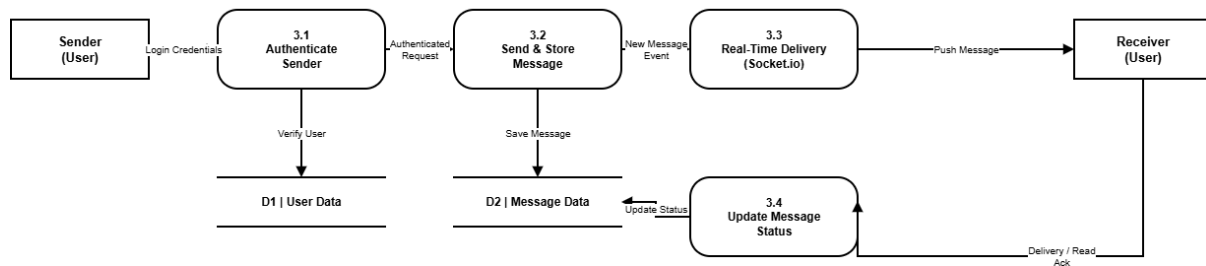


Level - 0

Figure 5.2 Data Flow Diagram

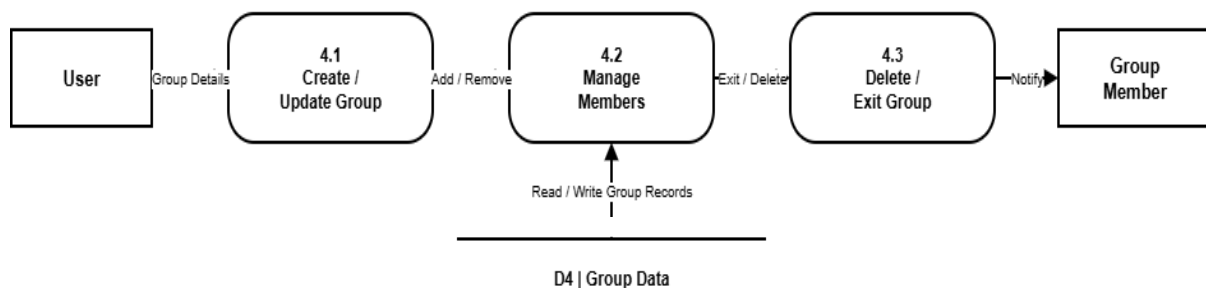


Level - 1



Level - 2 DFD Messaging

DFD Level 2 – Group Management (Process 4.0)



Level - 2 DFD Group Management

5.3 Class Diagram

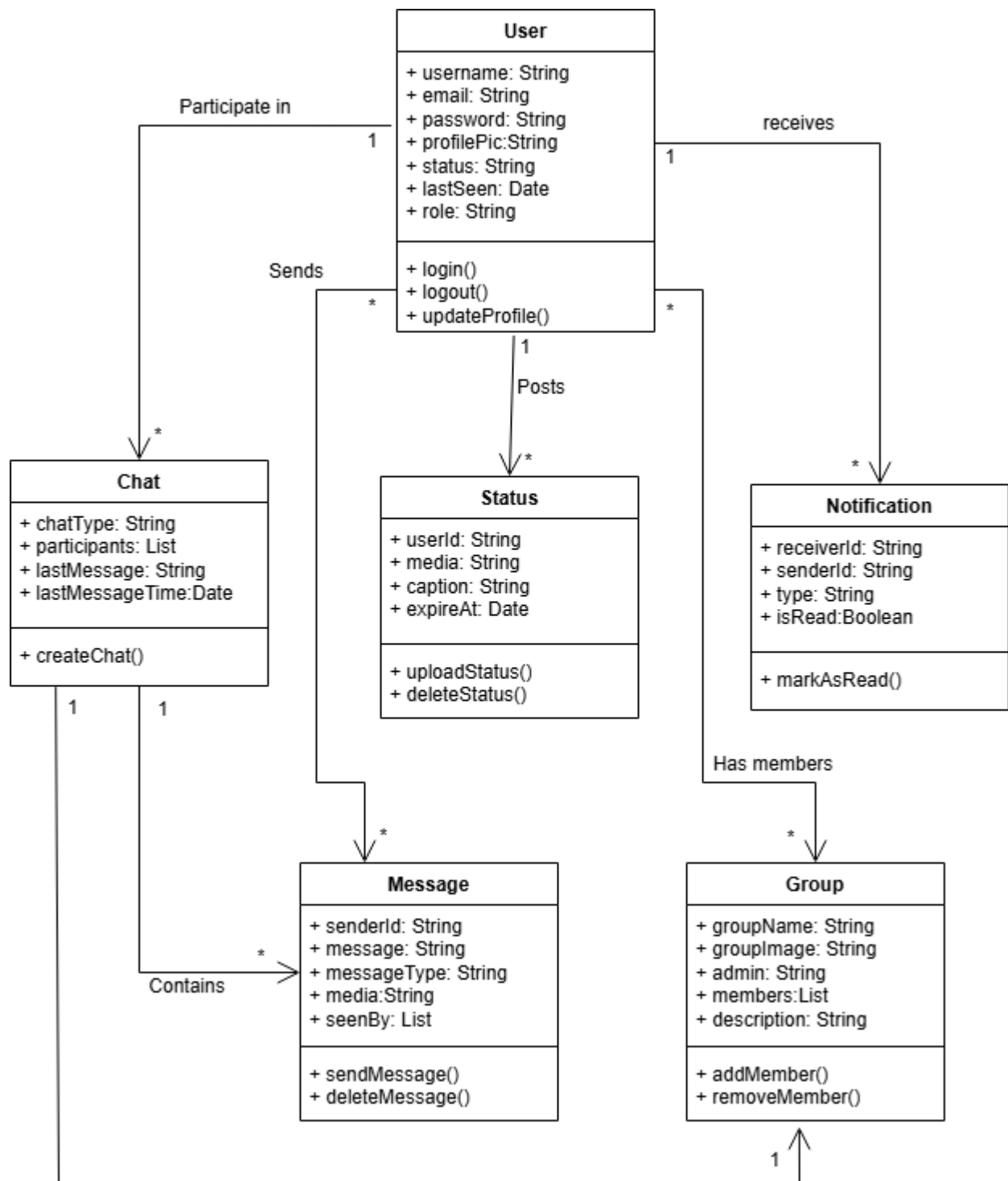


Figure 5.3 Class Diagram

5.4 Use Case Diagram

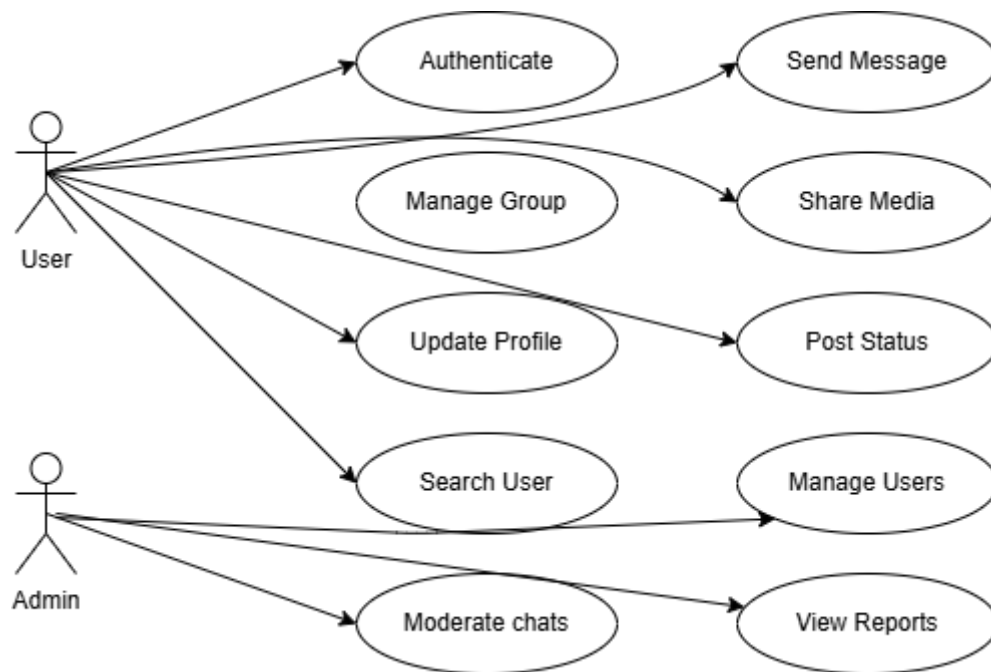


Figure 5.4 Use Case Diagram

5.5 Sequential Diagram

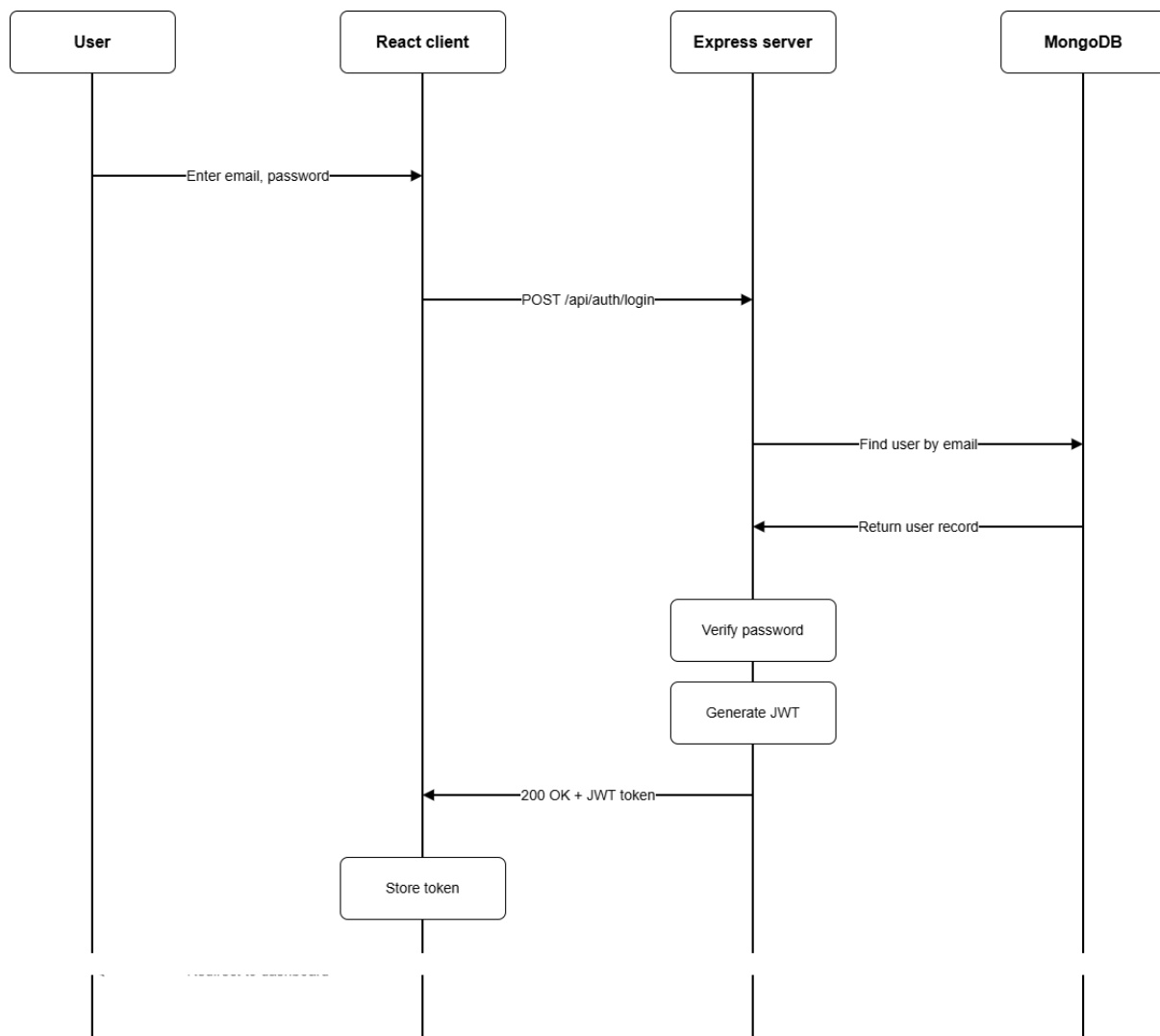


Figure 5.5 Sequential Diagram

5.6 Activity Diagram

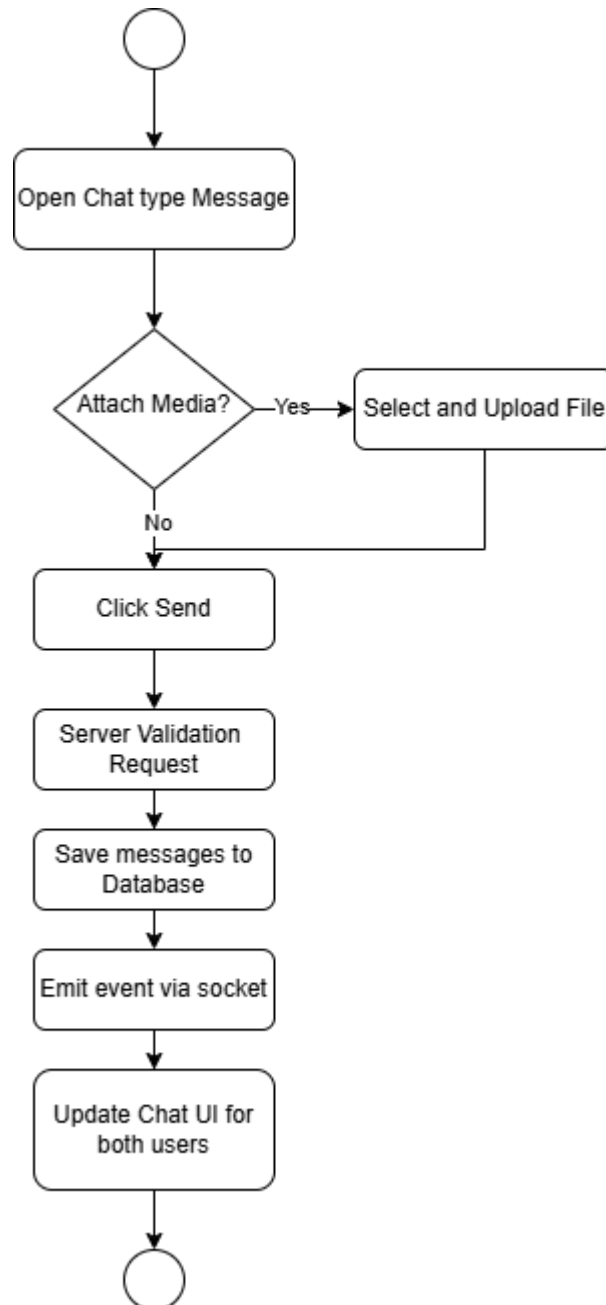


Figure 5.6 Activity Diagram

5.7 Screen Design

1. User Search Page

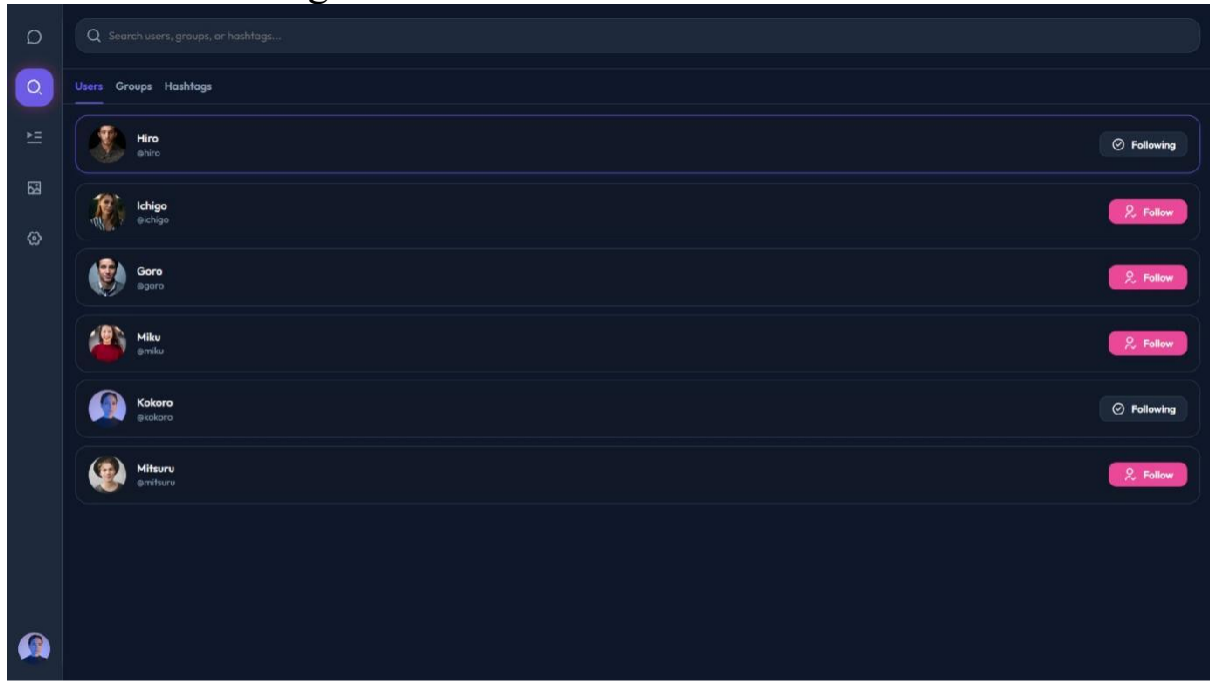


Figure 5.7 User Search Page

2. Status

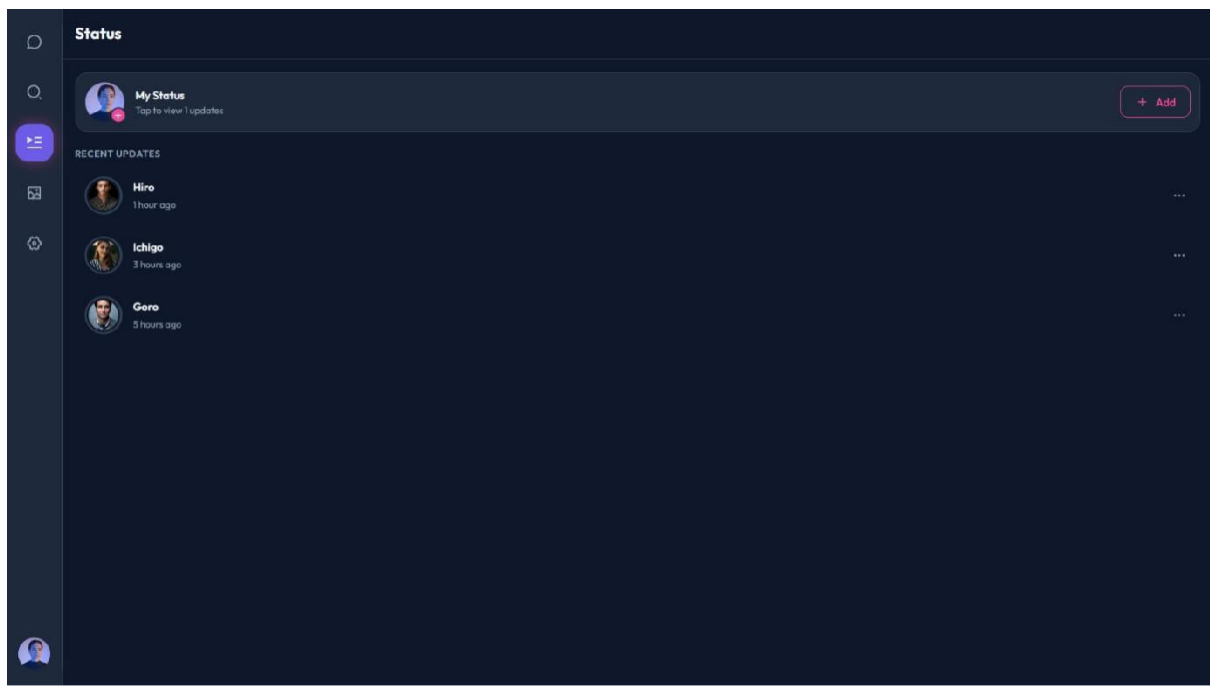


Figure 5.7.1 Status page

3. Media and Gallery

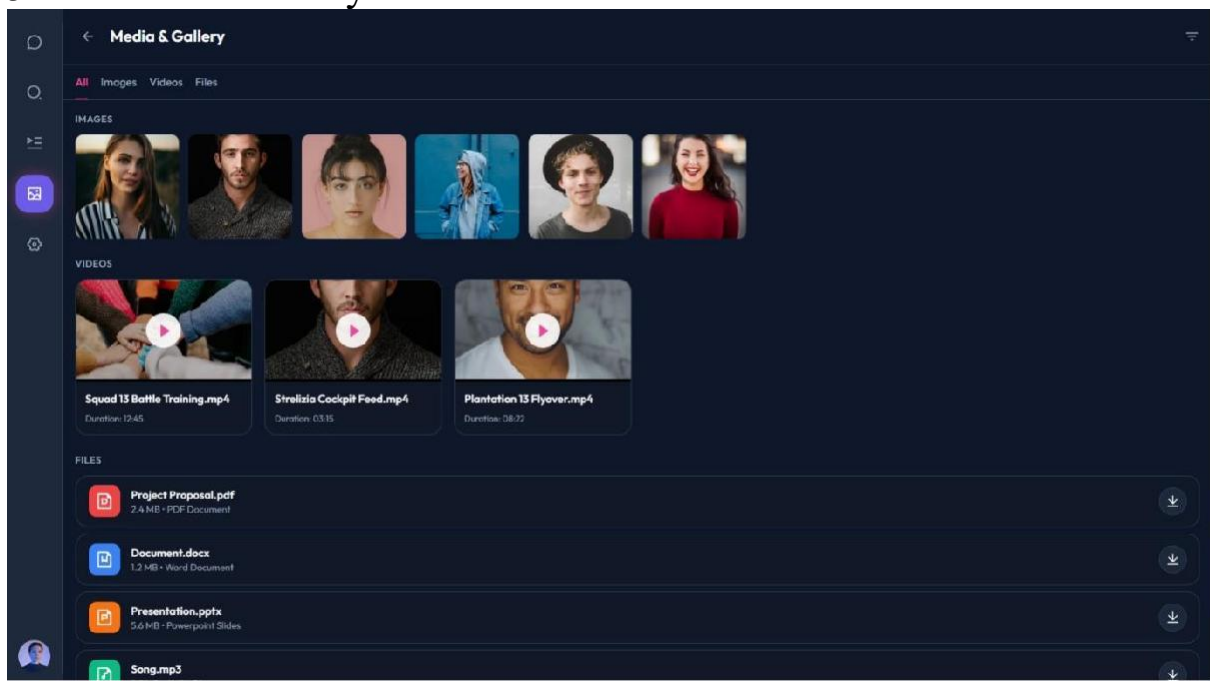


Figure 5.7.2 Media and Gallery page

4. Settings

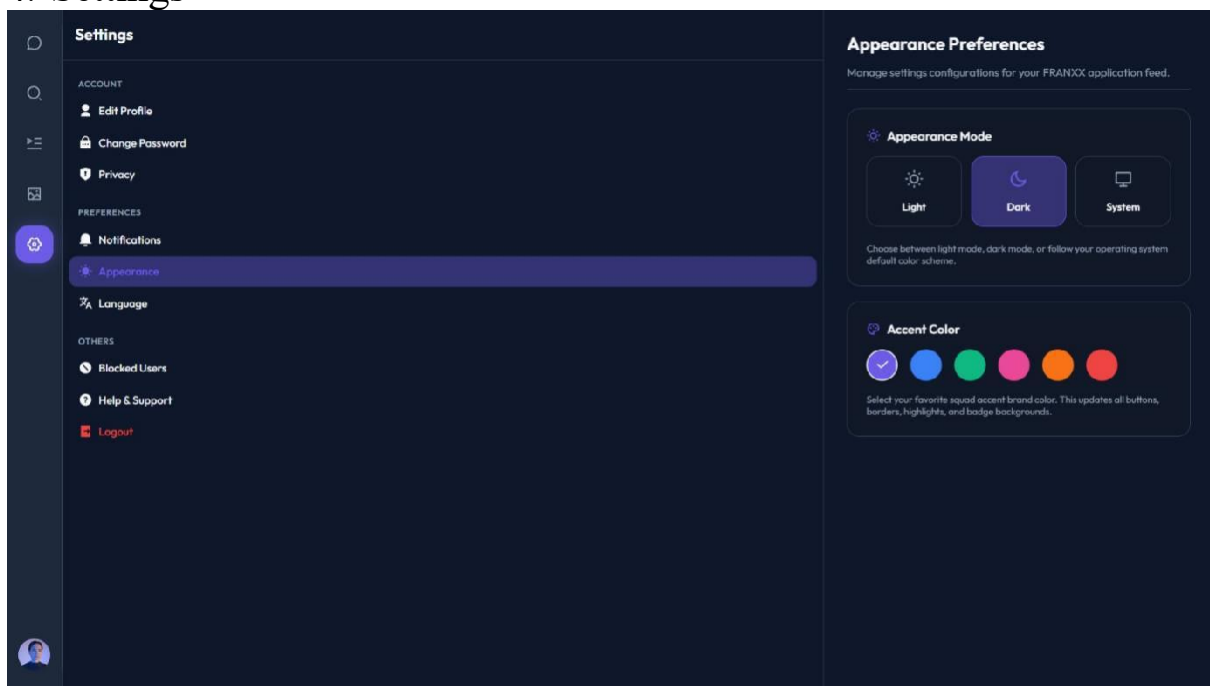


Figure 5.7.3 Settings page

5. Profile

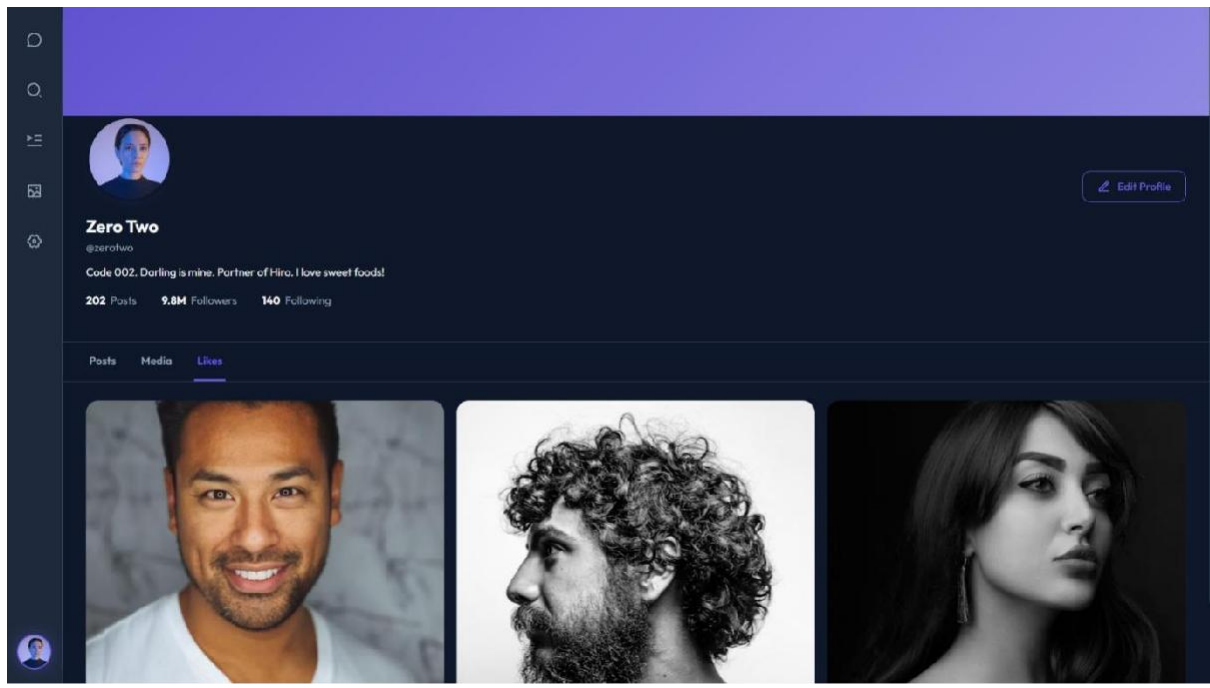


Figure 5.7.4 Profile page

6. Admin Dashboard

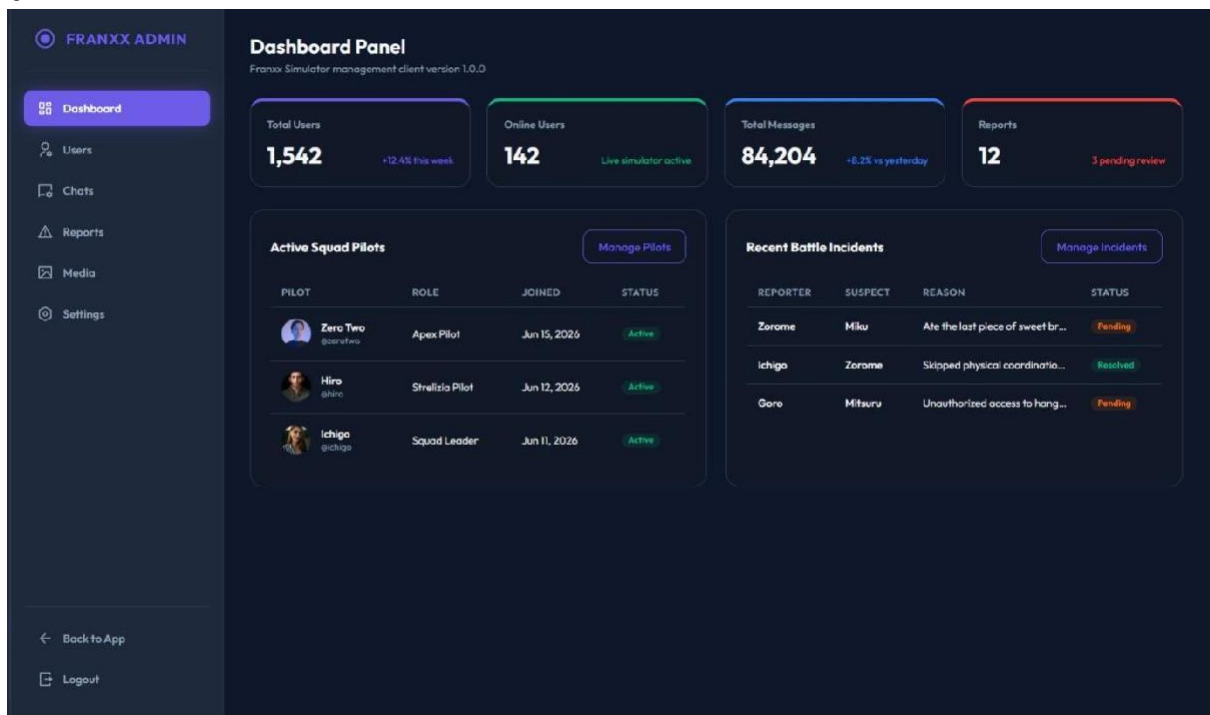


Figure 5.7.5 Admin Dashboard page

7. User Panel

FRANXX ADMIN

Dashboard

Users

Chats

Reports

Media

Settings

Back to App

Logout

Users Panel

Franxx Simulator management client version 1.0.0

Pilots Management

USER	SYSTEM ROLE	JOINED DATE	STATUS	ACTIONS
Zero Two @zerotwo	Apex Pilot	Jun 15, 2026	Active	Suspend
Hira @hira	Strelizia Pilot	Jun 12, 2026	Active	Suspend
Ichigo @ichigo	Squad Leader	Jun 11, 2026	Active	Suspend
Goro @goro	Delphinium Pilot	Jun 11, 2026	Offline	Activate
Zorome @zorome	Argentea Pilot	Jun 14, 2026	Active	Suspend

Figure 5.7.6 User Panel page

5.8 Database Structure

1. Users Collection

Field Name	Data Type	Key Constraint	Description
_id	ObjectId	Primary Key	Unique identifier for each user.
fullName	String	Required	User's full name.
username	String	Unique, Required	Unique username.
email	String	Unique, Required	User email address.
password	String	Required	Bcrypt hashed password.
profilePic	String	Optional	Profile image URL.
bio	String	Optional	User biography.
phone	String	Optional	Mobile number.
isOnline	Boolean	Default: false	User online status.
lastSeen	Date	System Generated	Last active timestamp.
createdAt	Date	System Generated	Account creation time.
updatedAt	Date	System Generated	Last update time.

Table 5.8 Users Collection

2. Chat Collection

Field Name	Data Type	Key Constraint	Description
_id	ObjectId	Primary Key	Unique chat ID.
chatType	String	Required	Private or Group.
participants	Array(ObjectId)	Required	List of participant IDs.
groupName	String	Optional	Group name.
groupImage	String	Optional	Group profile image.
admins	Array(ObjectId)	Optional	Group admin IDs.
createdBy	ObjectId	Foreign Key	User who created the group.
lastMessage	ObjectId	Foreign Key	Latest message ID.
createdAt	Date	System Generated	Chat creation date.
updatedAt	Date	System Generated	Last update date.

Table 5.8.1 Chat Collection

3.Messages Collection

Field Name	Data Type	Key Constraint	Description
_id	ObjectId	Primary Key	Unique message ID.
chatId	ObjectId	Foreign Key	Chat reference.
senderId	ObjectId	Foreign Key	Sender ID.
message	String	Optional	Text message.
messageType	String	Required	Text, Image, Video, Audio, File.
mediaUrl	String	Optional	Uploaded media URL.
replyTo	ObjectId	Optional	Reply message ID.
seenBy	Array(ObjectId)	Optional	Users who have seen the message.
deletedFor	Array(ObjectId)	Optional	Users who deleted the message.
createdAt	Date	System Generated	Message timestamp.
updatedAt	Date	System Generated	Last edit timestamp.

Table 5.8.2 Messages Collection

4.Status Collection

Field Name	Data Type	Key Constraint	Description
_id	ObjectId	Primary Key	Unique status ID.
userId	ObjectId	Foreign Key	User who posted status.
mediaUrl	String	Required	Image or video URL.
caption	String	Optional	Status caption.
viewers	Array(ObjectId)	Optional	Users who viewed the status.
expiresAt	Date	Required	Status expiry (24 hours).
createdAt	Date	System Generated	Upload timestamp.

Table 5.8.3 Status Collection

5. Notifications Collection

Field Name	Data Type	Key Constraint	Description
_id	ObjectId	Primary Key	Notification ID.
userId	ObjectId	Foreign Key	Receiver ID.
senderId	ObjectId	Foreign Key	Sender ID.
type	String	Required	Message, Group, Status.
title	String	Required	Notification title.
body	String	Required	Notification content.
isRead	Boolean	Default: false	Read status.
createdAt	Date	System Generated	Notification time.

Table 5.8.4 Notifications Collection

6. Settings Collection

Field Name	Data Type	Key Constraint	Description
_id	ObjectId	Primary Key	Setting ID.
userId	ObjectId	Foreign Key	User reference.
theme	String	Default: Light	App theme.
language	String	Default: English	Preferred language.
notifications	Boolean	Default: true	Notification preference.
privacy	String	Default: Everyone	Privacy setting.
lastSeenVisibility	String	Default: Everyone	Last seen visibility.
createdAt	Date	System Generated	Created timestamp.
updatedAt	Date	System Generated	Last updated timestamp.

Table 5.8.5 Settings Collection

6. SYSTEM TESTING

6.1 Registration & Login Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_01	Test user registration	Registration page should be accessible	1. Open sign up page 2. Enter details 3. Click sign up	Name, username, email, password	User account should be created	Account created successfully	Pass
Tc_02	Test user login	User should be registered	1. Open login page 2. Enter email/username 3. Enter password 4. Click login	Valid login details	User should login successfully	User logged in successfully	Pass
Tc_03	Test invalid login	Login page should be accessible	1. Enter invalid email 2. Enter incorrect password 3. Click login	Incorrect email/password	Error message should be displayed	Error message displayed	Pass
Tc_04	Test logout	User should be logged in	1. Open settings/profile 2. Click logout	Logged-in user	User should logout	User logged out successfully	Pass

Table 6 Registration & Login Testing

6.2 Private Chat Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_05	Test private chat	Two users should be connected	1. Select user 2. Open chat	Selected user	Private chat should open	Chat opened successfully	Pass
Tc_06	Test send message	Chat should be open	1. Type message 2. Click send	"hello"	Message should be sent	Message sent successfully	Pass
Tc_07	Test receive message	Both users should be online	1. User a sends message 2. Check user b	Text message	Message should appear in real time	Message received successfully	Pass
Tc_08	Test chat history	Previous messages should exist	1. Open existing chat	Existing conversation	Previous messages should be displayed	Chat history displayed	Pass
Tc_09	Test emoji	Chat should be open	1. Open emoji option 2. Select emoji 3. Send		Emoji should be sent	Emoji sent successfully	Pass
Tc_10	Test message seen status	Message should be sent	1. Send message 2. Receiver opens chat	Sent message	Seen status should update	Seen status updated	Pass

Table 6.1 Private Chat Testing

6.3 Chat Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_11	Test group creation	User should be logged in	1. Open groups 2. Create group 3. Add members	Group name, members	Group should be created	Group created successfully	Pass
Tc_12	Test group message	Group should exist	1. Open group 2. Type message 3. Send	"hello group"	All members should receive message	Message received by members	Pass
Tc_13	Test add member	Group admin required	1. Open group info 2. Select add member 3. Select user	Registered user	User should be added	Member added successfully	Pass
Tc_14	Test remove member	Group admin required	1. Open members 2. Select user 3. Remove	Group member	Member should be removed	Member removed successfully	Pass
Tc_15	Test group information	Group should exist	1. Open group 2. Click group header	Existing group	Group details should display	Group details displayed	Pass

Table 6.2 Chat Testing

6.4 User Search & Connection Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_16	Test user search	User should be logged in	1. Open search 2. Enter username	Username	Matching users should display	Users displayed successfully	Pass
Tc_17	Test invalid search	Search page should be open	1. Enter unknown username 2. Search	Invalid username	No user/result message should appear	No result displayed	Pass
Tc_18	Test user profile	User should appear in search	1. Search user 2. Click in profile	Registered user	User profile should open	Profile opened successfully	Pass
Tc_19	Test connection request	Another user should exist	1. Search user 2. Click add	Selected user	Request should be sent	Request sent successfully	Pass
Tc_20	Test accept request	Request should exist	1. Open notification 2. Accept request	Pending request	Users should be connected	Connection created successfully	Pass

Table 6.3 User Search & Connection Testing

6.5 User Profile Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_21	Test view profile	User should be logged in	1. Open profile	User account	Profile information should display	Profile displayed successfully	Pass
Tc_22	Test edit profile	Profile should exist	1. Click edit 2. Change details 3. Save	Name, about	Profile should update	Profile updated successfully	Pass
Tc_23	Test profile picture	User should be logged in	1. Edit profile 2. Select image 3. Save	Image file	Profile picture should update	Picture updated successfully	Pass
Tc_24	Test other user profile	Another user should exist	1. Search user 2. Open profile	Selected user	Other user's profile should display	Profile displayed successfully	Pass

Table 6.4 User Profile Testing

6.6 Status Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_25	Test text status	User should be logged in	1. Open status 2. Add text 3. Upload	"good morning"	Text status should upload	Status uploaded successfully	Pass
Tc_26	Test image status	Status page should be accessible	1. Select image 2. Upload	Image	Image status should upload	Image uploaded successfully	Pass
Tc_27	Test status visibility	Users should be connected	1. User a uploads status 2. User b opens status	User a's status	Connected user should see status	Status displayed successfully	Pass
Tc_28	Test status privacy	Users should not be connected	1. Upload status 2. Check from non-follower account	Status	Non-follower should not see status	Status not displayed	Pass
Tc_29	Test delete status	Status should exist	1. Open own status 2. Click delete	Existing status	Status should be deleted	Status deleted successfully	Pass

Table 6.5 Status Testing

6.7 Media & File Sharing Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_30	Test image sharing	Chat should be open	1. Click attachment 2. Select image 3. Send	Image	Image should be shared	Image shared successfully	Pass
Tc_31	Test document sharing	Chat should be open	1. Select document 2. Send	Pdf/document	File should be shared	File shared successfully	Pass
Tc_32	Test shared media	Shared media should exist	1. Open chat info 2. Open media	Shared files	Shared media should display	Media displayed successfully	Pass
Tc_33	Test file download	Shared file should exist	1. Select file 2. Click download	Shared file	File should download	File downloaded successfully	Pass

Table 6.6 Media & File Sharing Testing

6.8 Notification Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_34	Test message notification	Two users should exist	1. User a sends message 2. Check user b	New message	Notification should appear	Notification displayed	Pass
Tc_35	Test request notification	Users should not be connected	1. User a sends request 2. Check user b	Connection request	Request notification should appear	Notification displayed	Pass
Tc_36	Test group notification	User should be group member	1. Send group activity 2. Check member	Group activity	Group notification should appear	Notification displayed	Pass

Table 6.7 Notification Testing

6.9 Settings Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_37	Test change password	User should be logged in	1. Open settings 2. Change password 3. Save	Old and new password	Password should update	Password updated successfully	Pass
Tc_38	Test theme settings	Settings should be accessible	1. Open theme 2. Select theme	Dark/light	Theme should change	Theme changed successfully	Pass
Tc_39	Test privacy settings	User should be logged in	1. Open privacy 2. Change setting 3. Save	Privacy option	Setting should update	Setting updated successfully	Pass

Table 6.8 Settings Testing

6.10 Admin Panel Testing

Test case id	Test case objective	Pre-requisite	Steps	Input data	Expected output	Actual output	Status
Tc_40	Test admin login	Admin account should exist	1. Open admin login 2. Enter credentials 3. Login	Admin credentials	Admin dashboard should open	Dashboard opened successfully	Pass
Tc_41	Test view users	Admin should be logged in	1. Open user management	Registered users	All users should display	Users displayed successfully	Pass
Tc_42	Test search user	Admin should be logged in	1. Open users 2. Search user	Username	Matching user should display	User displayed successfully	Pass
Tc_43	Test block user	User should exist	1. Select user 2. Click block	Selected user	User should be blocked	User blocked successfully	Pass
Tc_44	Test delete user	User should exist	1. Select user 2. Click delete 3. Confirm	Selected user	User should be deleted	User deleted successfully	Pass
Tc_45	Test admin security	Normal user should be logged in	1. Enter admin dashboard url	Normal user	Access should be denied	Access denied successfully	Pass

Table 6.9 Admin Panel Testing

7. CONCLUSION

The FRANXX Real-Time Chat Application was developed to provide a simple and useful platform for online communication. The application allows registered users to search for other users, communicate through private and group chats, share media, upload status updates and manage their profiles.

The project helped in understanding how a real-time web application works from both the frontend and backend sides. Technologies such as React.js, Node.js, Express.js, MongoDB and Socket.IO were used together to develop the complete application.

The main objective of the project was to build a real-time communication system with useful messaging and social features. Most of the planned modules were successfully implemented and tested. The project can also be improved further by adding more advanced communication and security features in the future.

8. LEARNING DURING PROJECT WORK

During the development of the FRANXX project, I learned many practical concepts that were different from only studying programming theory.

I learned how frontend and backend applications communicate using APIs and how data can be stored and managed using MongoDB. I also understood how JWT authentication can be used to protect user accounts and application routes.

One of the important parts of this project was real-time communication. By working with Socket.IO, I learned how messages can be sent and received without refreshing the page. I also learned about online/offline status, typing indicators and real-time notifications.

While developing the project, I faced different problems related to APIs, database data, message handling, user authentication and frontend state. Finding and solving these problems improved my debugging and problem-solving skills.

The project also improved my knowledge of:

- React.js and component-based frontend development.
- Node.js and Express.js for backend development.
- MongoDB and database operations.
- REST API development and integration.
- JWT authentication and route protection.
- Socket.IO and real-time communication.
- Git and GitHub for source code management.

Overall, this project gave me practical experience in developing a complete MERN stack application.

8.1 Future Enhancement

FRANXX currently provides the main features required for a real-time chat application. However, some additional features can be added in future versions.

- Voice Calling: Users can communicate through real-time voice calls.
- Video Calling: One-to-one and group video calling can be provided.
- Message Reactions: Users can react to messages using emojis.
- Reply and Forward: Messages can be replied to and forwarded to other chats.
- Message Editing: Users can edit already sent messages within a limited time.
- Advanced Privacy: More options can be provided for last seen, profile picture and status visibility.
- Push Notifications: Users can receive notifications even when the application is not currently open.
- End-to-End Encryption: Advanced encryption can be implemented to improve message privacy.
- Voice Messages: Users can record and send voice messages.
- Mobile Application: A mobile version of FRANXX can be developed for Android and iOS.
- Cloud Storage: Shared images, videos and documents can be stored using a cloud storage service.
- Advanced Admin Monitoring: More detailed reports and system activity information can be provided to administrators.

These improvements can make FRANXX more useful, secure and suitable for a larger number of users.

9. Bibliography

9.1 Online References

- [1] *MongoDB Inc., “MongoDB Documentation,” [online]. Available: <https://www.mongodb.com/docs/>. [Accessed: July 10, 2026].*
- [2] *React, “React Documentation,” [online]. Available: <https://react.dev/>. [Accessed: June 20, 2026].*
- [3] *Node.js, “Node.js Documentation,” [online]. Available: <https://nodejs.org/docs/latest/api/>. [Accessed: July 25, 2026].*
- [4] *Express.js, “Express Documentation,” [online]. Available: <https://expressjs.com/>. [Accessed: August 5, 2026].*
- [5] *MDN Web Docs, “JavaScript Documentation,” [online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. [Accessed: August 14, 2026].*

9.2 Offline References

Text Book:

- [1] *Software Engineering – A Practitioner’s Approach, Roger S. Press-man, McGraw Hill, 7th Edition.*
- [2] *Unified Modeling Language – User’s Guide, Grady Booch, James Rumbaugh, Ivar Jacobson, Addison – Wesley, 2nd Edition*
- [3] *Object Oriented Modeling and Design using UML, Michael Blaha and James Rumbaugh, Pearson Education, 2nd edition*